

Федеральное государственное автономное образовательное учреждение высшего образования «Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Направление подготовки: 09.03.04 – Системное и прикладное программное обеспечение

Дисциплина «Алгоритмы и структуры данных»

Отчёт по блоку задач №3

Выполнил
Линейский Аким Евгеньевич
Р3215
Преподаватель
Тараканов Денис Сергеевич

Санкт-Петербург, 2026

1. Задача I. Машины

1.1. Условие задачи

I. Машины

✓ Полное решение

Ограничение времени	1 секунда
Ограничение памяти	256 Мб
Ввод	стандартный ввод или input.txt
Вывод	стандартный вывод или output.txt

Петя, которому три года, очень любит играть с машинками. Всего у Пети N различных машинок, которые хранятся на полке шкафа так высоко, что он сам не может до них дотянуться. Одновременно на полу комнаты может находиться не более K машинок. Петя играет с одной из машинок на полу и если он хочет поиграть с другой машинкой, которая также находится на полу, то дотягивается до нее сам. Если же машинка находится на полке, то он обращается за помощью к маме. Мама может достать для Пети машинку с полки и одновременно с этим поставить на полку любую машинку с пола. Мама очень хорошо знает своего ребенка и может предугадать последовательность, в которой Петя захочет играть с машинками. При этом, чтобы не мешать Петиней игре, она хочет совершить как можно меньше операций по подъему машинок с пола, каждый раз правильно выбирая машинку, которую следует убрать на полку. Ваша задача состоит в том, чтобы определить минимальное количество операций. Перед тем, как Петя начал играть, все машинки стоят на полке.

Формат ввода

В первой строке содержатся три числа N, K и P ($1 \leq K, N \leq 100000, 1 \leq P \leq 500000$). В следующих P строках записаны номера машинок в том порядке, в котором Петя захочет играть с ними.

Формат вывода

Выведите единственное число: минимальное количество операций, которое надо совершить Петиней маме.

Пример

Ввод 	Вывод 
3 2 7 1 2 3 1 3 1 2	4

Примечания

В этой задаче можно использовать STL.

Пояснения к примеру:

Операция 1: снять машинку 1

Операция 2: снять машинку 2

Операция 3: поднять машинку 2 и снять машинку 3

1.2. Идея решения

Для минимизации количества подходов мамы к полке используется жадный алгоритм, напоминающий стратегию вытеснения кэша по принципу наиболее удаленного использования в будущем.

Сначала для каждой машинки строится очередь из индексов моментов времени, когда Петя захочет с ней поиграть. На полу поддерживается множество машинок текущего состава, размер которого ограничен величиной K . При обработке очередного запроса, если запрашиваемая машинка уже находится на полу, мы просто обновляем время её следующего появления в приоритетном множестве. Если машинки на полу нет, происходит добавление, увеличивающее счетчик маминых усилий. Если при этом пол уже заполнен, из него удаляется та машинка, которая потребуется Пете позже всех остальных. Для быстрого поиска этой машинки используется сбалансированное дерево поиска `std::set`, хранящее пары из времени следующего запроса и идентификатора машинки.

1.3. Исходный код программы (C++)

```
1 #include <iostream>
2 #include <queue>
3 #include <set>
4 #include <vector>
5
6 int main() {
7     std::ios_base::sync_with_stdio(false);
8     std::cin.tie(nullptr);
9
10    int N, K, P;
11    if (!(std::cin >> n >> K >> P)) {
12        return 0;
13    }
14
15    std::vector<int> petyas_wishlist(P);
16    std::vector<std::queue<int>> future_dates(N + 1);
17
18    for (int i = 0; i < P; ++i) {
19        std::cin >> petyas_wishlist[i];
20        future_dates[petyas_wishlist[i]].push(i);
21    }
22
23    std::set<std::pair<int, int>> floor_party;
24    std::vector<bool> is_on_floor(N + 1, false);
25    int total_mom_efforts = 0;
26
27    for (int i = 0; i < P; ++i) {
28        int current_car = petyas_wishlist[i];
29        future_dates[current_car].pop();
30
31        int next_time_petya_screams =
32            future_dates[current_car].empty() ? P + 1 : future_dates[
current_car].front();
33
34        if (is_on_floor[current_car]) {
35            floor_party.erase(floor_party.find({i, current_car}));
36            floor_party.insert({next_time_petya_screams, current_car});
37        } else {
38            if (static_cast<int>(floor_party.size()) == K) {
39                auto longest_idle_car = prev(floor_party.end());
40                is_on_floor[longest_idle_car->second] = false;
```

```

41         floor_party.erase(longest_idle_car);
42     }
43     is_on_floor[current_car] = true;
44     floor_party.insert({next_time_petya_screams, current_car});
45     total_mom_efforts++;
46 }
47 }
48
49 std::cout << total_mom_efforts << std::endl;
50 return 0;
51 }

```

2. Задача J. Гоблины и шаманы

2.1. Условие задачи

J. Гоблины и очереди

✓ Полное решение

Ограничение времени	0.6 секунд
Ограничение памяти	256 Мб
Ввод	стандартный ввод или input.txt
Вывод	стандартный вывод или output.txt

Гоблины Мглистых гор очень любят ходить к своим шаманам. Так как гоблинов много, к шаманам часто образуются очень длинные очереди. А поскольку много гоблинов в одном месте быстро образуют шумную толку, которая мешает шаманам проводить сложные медицинские манипуляции, последние решили установить некоторые правила касательно порядка в очереди.

Обычные гоблины при посещении шаманов должны вставать в конец очереди. Привилегированные же гоблины, знающие особый пароль, встают ровно в ее середину, причем при нечетной длине очереди они встают сразу за центром.

Так как гоблины также широко известны своим непочтительным отношением ко всяческим правилам и законам, шаманы попросили вас написать программу, которая бы отслеживала порядок гоблинов в очереди.

Формат ввода

В первой строке входных данных записано число N ($1 \leq N \leq 10^5$) — количество запросов. Следующие N строк содержат описание запросов в формате:

- $+i$ — гoblin с номером i ($1 \leq i \leq N$) встает в конец очереди.
- $*i$ — привилегированный гoblin с номером i встает в середину очереди.
- $-$ — первый гoblin из очереди уходит к шаманам. Гарантируется, что на момент такого запроса очередь не пуста.

Формат вывода

Для каждого запроса типа $-$ программа должна вывести номер гоблина, который должен уйти к шаманам.

2.2. Идея решения

Для эффективного моделирования очереди со вставкой элементов в середину структура данных разделяется на две независимые части. Мы используем два двусторонних конца `std::deque`. Первая структура `head` хранит первую половину очереди, а вторая структура `tail` отвечает за вторую половину.

Для поддержания корректного состояния после каждой операции вызывается метод балансировки `balance`. Правило инварианта требует, чтобы размер `head` был равен размеру `tail` либо превышал его ровно на один элемент. Обычные гоблины со знаком плюс добавляются в конец `tail`. Привилегированные гоблины со знаком звездочка вставляются в начало `tail`, что благодаря свойству балансировки как раз соответствует середине общей очереди. Извлечение элементов со знаком минус всегда производится из начала `head`. Все операции выполняются за константное время.

2.3. Исходный код программы (C++)

```
1 #include <deque>
2 #include <iostream>
3
4 struct GoblinQueue {
5     std::deque<int> head;
6     std::deque<int> tail;
7
8     void balance() {
9         if (head.size() < tail.size()) {
10             head.push_back(tail.front());
11             tail.pop_front();
12         } else if (head.size() > tail.size() + 1) {
13             tail.push_front(head.back());
14             head.pop_back();
15         }
16     }
17
18     void push_back(int id) {
19         tail.push_back(id);
20         balance();
21     }
22
23     void push_mid(int id) {
24         tail.push_front(id);
25         balance();
26     }
27
28     int pop_front() {
29         int id = head.front();
30         head.pop_front();
31         balance();
32         return id;
33     }
34 };
35
36 int main() {
37     std::ios_base::sync_with_stdio(false);
38     std::cin.tie(nullptr);
39
40     int n_queries;
41     if (!(std::cin >> n_queries))
42         return 0;
43
44     GoblinQueue shaman_line;
45
46     while (n_queries--) {
47         char op_type;
48         std::cin >> op_type;
49
50         if (op_type == '+') {
51             int id;
52             std::cin >> id;
53             shaman_line.push_back(id);
54         } else if (op_type == '*') {
55             int id;
56             std::cin >> id;
57             shaman_line.push_mid(id);
58         } else if (op_type == '-') {
```

```
59     std::cout << shaman_line.pop_front() << "\n";
60 }
61 }
62
63 return 0;
64 }
```

3. Задача К. Менеджер памяти

3.1. Условие задачи

К. Менеджер памяти-1

✓ Полное решение

Ограничение времени	1 секунда
Ограничение памяти	64 Мб
Ввод	стандартный ввод или input.txt
Вывод	стандартный вывод или output.txt

Пете поручили написать менеджер памяти для новой стандартной библиотеки языка $\varphi++$. В распоряжении у менеджера находится массив из N последовательных ячеек памяти, пронумерованных от 1 до N . Задача менеджера – обрабатывать запросы приложений на выделение и освобождение памяти. Запрос на выделение памяти имеет один параметр K . Такой запрос означает, что приложение просит выделить ему K последовательных ячеек памяти. Если в распоряжении менеджера есть хотя бы один свободный блок из K последовательных ячеек, то он обязан в ответ на запрос выделить такой блок. При этом непосредственно перед самой первой ячейкой памяти выделяемого блока не должно располагаться свободной ячейки памяти. После этого выделенные ячейки становятся занятыми и не могут быть использованы для выделения памяти, пока не будут освобождены. Если блока из K последовательных свободных ячеек нет, то запрос отклоняется. Запрос на освобождение памяти имеет один параметр T . Такой запрос означает, что менеджер должен освободить память, выделенную ранее при обработке запроса с порядковым номером T . Запросы нумеруются, начиная с единицы. Гарантируется, что запрос с номером T – запрос на выделение, причем к нему еще не применялось освобождение памяти. Освобожденные ячейки могут снова быть использованы для выделения памяти. Если запрос с номером T был отклонен, то текущий запрос на освобождение памяти игнорируется. Требуется написать менеджер памяти, удовлетворяющий приведенным критериям.

Формат ввода

Первая строка входного файла содержит числа N и M – количество ячеек памяти и количество запросов соответственно ($1 \leq N \leq 2^{31} - 1$; $1 \leq M \leq 10^5$). Каждая из следующих M строк содержит по одному числу: $(i+1)$ -я строка входного файла ($1 \leq i \leq M$) содержит либо положительное число K , если i -й запрос – запрос на выделение с параметром K ($1 \leq K \leq N$), либо отрицательное число $-T$, если i -й запрос – запрос на освобождение с параметром T ($1 \leq T < i$).

Формат вывода

Для каждого запроса на выделение памяти выведите в выходной файл результат обработки этого запроса: для успешных запросов выведите номер первой ячейки памяти в выделенном блоке, для отклоненных запросов выведите число -1 . Результаты нужно выводить в порядке следования запросов во входном файле.

3.2. Идея решения

Для реализации менеджера памяти необходимо оперативно искать свободные блоки подходящего размера и объединять соседние освободившиеся участки. Задача решается параллельным ведением двух упорядоченных множеств свободных блоков.

Множество `by_size` сортирует свободные фрагменты по их размеру для быстрого поиска подходящего места методом `lower_bound`. Множество `by_addr` сортирует те же самые блоки по их начальному адресу в памяти, что критически важно при освобождении. При

поступлении запроса на выделение памяти ищется минимальный достаточный блок. Если его размер превышает запрошенный, остаток блока пересчитывается и возвращается обратно в структуры данных. При поступлении запроса на освобождение по индексу из истории извлекаются параметры блока. Затем через `by_addr` проверяется наличие свободных соседей непосредственно слева и справа. В случае обнаружения смежных свободных участков они сливаются в один непрерывный блок большей длины.

3.3. Исходный код программы (C++)

```
1 #include <iostream>
2 #include <map>
3 #include <set>
4 #include <vector>
5
6 using namespace std;
7
8 struct Chunk {
9     int start;
10    int size;
11
12    bool operator< (const Chunk &other) const {
13        if (size != other.size) {
14            return size < other.size;
15        }
16        return start < other.start;
17    }
18 };
19
20 struct AddrComp {
21     bool operator()(const Chunk &a, const Chunk &b) const {
22         return a.start < b.start;
23     }
24 };
25
26 set<Chunk> by_size;
27 set<Chunk, AddrComp> by_addr;
28 map<int, Chunk> history;
29
30 void add_block(int s, int n) {
31     if (n <= 0) {
32         return;
33     }
34     Chunk c = {s, n};
35     by_size.insert(c);
36     by_addr.insert(c);
37 }
38
39 void del_block(int s, int n) {
40     Chunk c = {s, n};
41     by_size.erase(c);
42     by_addr.erase(c);
43 }
44
45 int main() {
46     ios_base::sync_with_stdio(false);
47     cin.tie(NULL);
48     int n, m;
49     if (!(cin >> n >> m)) {
50         return 0;
```

```

51 }
52 add_block(1, n);
53 for (int i = 1; i <= m; ++i) {
54     int req;
55     cin >> req;
56     if (req > 0) {
57         auto it = by_size.lower_bound({0, req});
58         if (it == by_size.end()) {
59             cout << "-1\\n";
60             history[i] = {0, 0};
61         } else {
62             Chunk found = *it;
63             del_block(found.start, found.size);
64             cout << found.start << "\\n";
65             history[i] = {found.start, req};
66             if (found.size > req) {
67                 add_block(found.start + req, found.size - req);
68             }
69         }
70     } else {
71         Chunk old = history[-req];
72         if (old.size > 0) {
73             int cur_s = old.start;
74             int cur_n = old.size;
75             auto it_up = by_addr.lower_bound({cur_s, cur_n});
76             if (it_up != by_addr.end() && it_up->start == cur_s + cur_n) {
77                 cur_n += it_up->size;
78                 del_block(it_up->start, it_up->size);
79             }
80             auto it_down = by_addr.lower_bound({cur_s, cur_n});
81             if (it_down != by_addr.begin()) {
82                 it_down--;
83                 if (it_down->start + it_down->size == cur_s) {
84                     cur_s = it_down->start;
85                     cur_n += it_down->size;
86                     del_block(it_down->start, it_down->size);
87                 }
88             }
89             add_block(cur_s, cur_n);
90         }
91     }
92 }
93 return 0;
94 }

```

4. Задача L. Минимум на отрезке

4.1. Условие задачи

L. Минимум на отрезке

✓ Полное решение

Ограничение времени	0.5 секунд
Ограничение памяти	256 Мб
Ввод	стандартный ввод или input.txt
Вывод	стандартный вывод или output.txt

Рассмотрим последовательность целых чисел длины N . По ней с шагом 1 двигается «окно» длины K , то есть сначала в «окне» видно первые K чисел, на следующем шаге в «окне» уже будут находиться K чисел, начиная со второго, и так далее до конца последовательности. Требуется для каждого положения «окна» определить минимум в нём.

Формат ввода

В первой строке входных данных содержатся два числа N и K ($1 \leq N \leq 150000$, $1 \leq K \leq 10000$, $K \leq N$) – длины последовательности и «окна», соответственно. На следующей строке находятся N чисел – сама последовательность. Числа последовательности не превосходят по модулю 10^5 .

Формат вывода

Выходные данные должны содержать $N - K + 1$ строк – минимумы для каждого положения «окна».

Пример

Ввод

```
7 3
1 3 2 4 5 3 1
```

Вывод

```
1 2 2 3 1
```

[Скачать условие задачи](#)

4.2. Идея решения

Для нахождения минимумов на всех сдвигающихся отрезках фиксированной длины за линейное время применяется алгоритм скользящего окна с использованием монотонной деки. В двусвязной очереди `std::deque` хранятся индексы элементов массива, причем значения элементов по этим индексам всегда строго возрастают от начала деки к её концу.

В ходе итерации по массиву сначала удаляется индекс с начала деки, если он вышел за левую границу текущего окна шириной k . Затем с конца деки один за другим удаляются индексы всех элементов, значения которых больше либо равны новому пришедшему элементу, поскольку они больше никогда не смогут стать минимумом в текущем или последующих окнах. После очистки новый индекс добавляется в конец структуры. Текущий минимум всегда располагается в начале монотонной деки и выводится на экран, как только правая граница итератора доходит до размера первого полноценного окна.

4.3. Исходный код программы (C++)

```

1 #include <deque>
2 #include <iostream>
3 #include <vector>
4
5 using namespace std;
6
7 int main() {
8     ios_base::sync_with_stdio(false);
9     cin.tie(NULL);
10    int length;
11    int window;
12    if (!(cin >> length >> window)) {
13        return 0;
14    }
15    vector<int> numbers(length);
16    for (int i = 0; i < length; ++i) {
17        cin >> numbers[i];
18    }
19    deque<int> indices_of_tiny_things;
20    for (int i = 0; i < length; ++i) {
21        if (!indices_of_tiny_things.empty() &&
22            indices_of_tiny_things.front() <= i - window) {
23            indices_of_tiny_things.pop_front();
24        }
25        while (!indices_of_tiny_things.empty() &&
26            numbers[indices_of_tiny_things.back()] >= numbers[i]) {
27            indices_of_tiny_things.pop_back();
28        }
29        indices_of_tiny_things.push_back(i);
30        if (i >= window - 1) {
31            cout << numbers[indices_of_tiny_things.front()]
32                << (i == length - 1 ? "" : " ");
33        }
34    }
35    cout << endl;
36    return 0;
37 }

```